



Comparative analysis of CatBoost, LightGBM, XGBoost, RF, and DT methods optimised with PSO to estimate the number of k-barriers for intrusion detection in wireless sensor networks

Kadir Ileri¹

Received: 7 December 2023 / Accepted: 11 April 2025 / Published online: 6 May 2025
© The Author(s) 2025

Abstract

The protection of borders is a critical concern for all countries, and Wireless Sensor Networks (WSNs) play a crucial role in assuring security by enabling intrusion detection and surveillance at border regions. This study presents an effective machine learning model designed to predict the number of k-barriers for rapid and robust intrusion detection and prevention in a rectangular area utilizing features extracted from a WSN through Monte-Carlo simulation. The proposed model is implemented using the CatBoost method, which is optimized with the metaheuristic Particle Swarm Optimisation (PSO) algorithm. To ensure a fair assessment, the performance of the proposed model is compared with state-of-the-art methods such as LightGBM, XGBoost, Random Forest (RF), and Decision Tree (DT) using metrics like MAE, MSE, RMSE, and R^2 . The proposed model demonstrates clear superiority, outperforming XGBoost, LightGBM, RF, and DT with an impressive R^2 value of 0.9998, MAE value of 0.6298, MSE value of 0.6018, and RMSE value of 0.7758. Additionally, the proposed model is evaluated against other existing methods that use the same dataset, and it produces the most favourable outcomes. Furthermore, the feature importance analysis reveals the network area and transmission range as the most influential features in predicting the number of k-barriers. In conclusion, the proposed CatBoost-PSO model exhibits exceptional performance and prediction accuracy, making it a promising approach for intrusion detection and prevention in WSN applications.

Keywords CatBoost · LightGBM · XGBoost · Intrusion detection · Wireless sensor networks

Abbreviations

CatBoost	Categorical Boosting
D1	Device 1
D2	Device 2
DT	Decision Tree
GPR	Gaussian Process Regression
GPS	Global Positioning System
LightGBM	Light Gradient Boosting Machine
M	Mobile
MAE	Mean Absolute Error
MDI	Mean Decrease Impurity
MSE	Mean Squared Error
NS	Network Simulator
PCA	Principal Component Analysis
PSO	Particle Swarm Optimisation

R^2	Coefficient of Determination
RF	Random Forest
RMSE	Root Mean Squared Error
S	Stationary
TBS	Target Based Statistics
UAV	Unmanned Aerial Vehicle
WSN	Wireless Sensor Network
XGBoost	Extreme Gradient Boosting

1 Introduction

The protection of borders is a very critical concern for all the countries in the world. To ensure the security of their own citizens, every nation is responsible for taking measures against unauthorized refugee influx, attempts by enemy soldiers to clandestinely steal national security information, the entry of drug smugglers, and other prohibited intrusions [1]. Many countries depend on their regular armies to safeguard their international borders from potential threats. However, conventional and periodic patrolling approaches come with

✉ Kadir Ileri
kileri@bandirma.edu.tr

¹ Electrical & Electronics Engineering Department, Bandirma Onyedi Eylul University, Balikesir, Turkey

inherent limitations. Due to international boundaries being long, it is not feasible to emplace soldiers to every point. This leaves ungarded areas vulnerable to enemy infiltration. To address these security challenges, accurately and rapidly identifying intruders and unauthorized activities is crucial. To accomplish monitoring and surveillance along international borders, the WSNs can be employed effectively [2–4].

The WSNs are extensively adopted and consist of sensors capable of processing, sensing, and transmitting data. Due to their advantages such as rapid and simple installation, cost-effectiveness, and the absence of a need for pre-existing infrastructure, the WSNs are widely used for intrusion detection purposes [5]. A WSN can be implemented to create a sensor barrier for any potential intrusion path of an intruder in the rectangular region, as illustrated in Fig. 1. Apart from intrusion detection, WSNs are used in various fields such as health monitoring, environment monitoring, landmine detection, fire detection, and so on [6–8].

Table 1 provides a concise summary of the contributions and key findings from various studies related to applications of barrier coverage using WSNs [9–20]. For each work, it

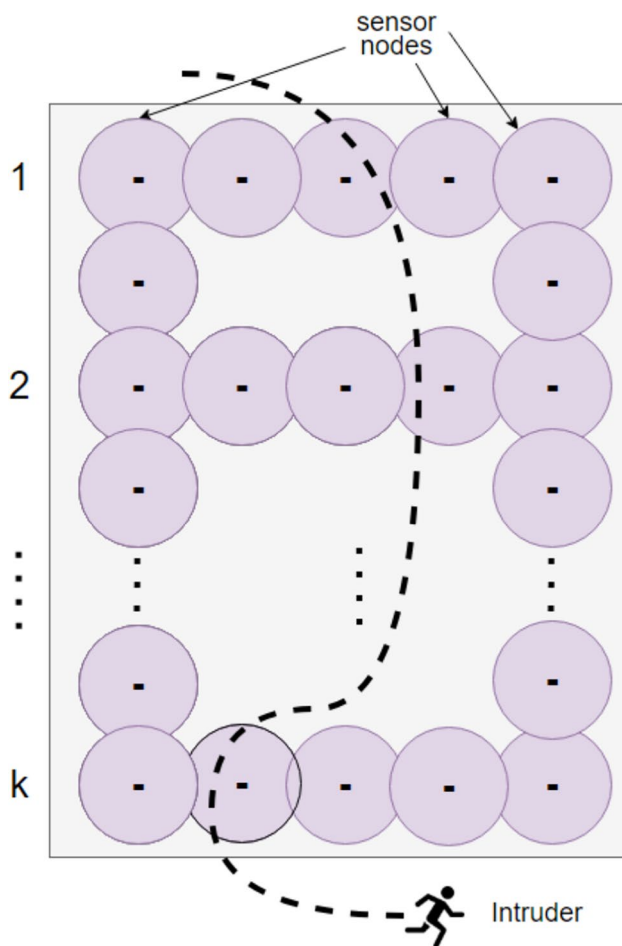


Fig. 1 Example of k -barrier coverage for an intruder

typically includes the name of the authors, publication year, short description, limitations, main outcomes, and sensor types used as mobile (M) or stationary (S). In one of these studies, Yang et al. [9] introduced an energy-efficient border intrusion detection system with the aim of minimizing intruder entries and improving detection accuracy. The focus of the research is on coverage quality and the techniques to improve it. The study proposes a repair method to identify and fix weak zones to ensure the desired barrier coverage quality. Additionally, a one-directional coverage model is presented for intrusion detection in border regions, including both single and a few intruder scenarios. In addition to these advantages, the proposed algorithm extends the lifetime of the WSN and maintains energy efficiency. Similarly, Raza et al. [10] investigated the performance of an intrusion detection system in a heterogeneous WSN. The investigation is done for WSNs formed through both Gaussian and uniform distribution of sensors. The impacts of sensor node densities on both distributions are analysed using k -sensing detection. Simulation results indicate that the heterogeneous WSN outperforms homogeneous WSN in intrusion detection. Sharma et al. [11] represented analytic statements to compute the k -barrier coverage probability in a mobile WSN for an intruder moving within the rectangular region. Various system and network parameters such as sensing range, sensor density, and intruder path angle are analysed for their impacts on the coverage probability. The research shows that the k -barrier coverage probability heightens with the intruder's path angle. It leads to higher detection probability as the intruder spends more time within the region.

In another study, Singh et al. [12] presented a comprehensive framework for accurately predicting k -barrier coverage probability for intruder detection. The three Gaussian Process Regression (GPR) based machine learning methods (GPR, S-GPR, and C-GPR) are implemented for the prediction. Simulation results show that the native GPR model performs the best, accurately predicting k -barrier coverage probability compared to S-GPR, and C-GPR. Further, Nurellari et al. [13] designed optimal trajectories of mobile sensors for intruder detection. These trajectories maximised the coverage area and reduced the consumption of energy to extend the lifetime of the sensor network. The performance of the sensor network is optimised by carefully designing trajectories and dynamically adapting them to changes for better intrusion detection outcomes. Besides, Saraereh et al. [14] proposed a method for secondary deployment of mobile sensors to increase the utilization rate. The set-based maximum flow method is used to compute the number of weak points in the network, and the binary search method is employed for the optimisation of the method. The primary objective of the research is to strengthen the weak points while minimizing the total movement distance of mobile sensors. Nevertheless, certain deficiencies persist

Table 1 Summary of related works

Reference	Year	Description	Advantages	Limitations	M/S
Yang et al. [9]	2014	Identifying weak zones to deal with bad coverage quality	- energy efficient detection	- one-directional coverage	S
Raza et al. [10]	2015	Analysing the influence of various sensor node densities in computing detection probability in a heterogeneous network	- operates in both Gaussian and uniform distribution scenarios	- analysed only for a single intruder	S
Sharma et al. [11]	2020	Analysing the analytic statements for computing the k-barrier coverage probability in a mobile WSN for an intruder	- analysed for different path types of an intruder	- not analysed for different mobility and sensing models	M
Singh et al. [12]	2021	An implementation of three machine learning methods (GPR, S-GPR, and C-GPR) to estimate the k-barrier coverage probability for intrusion detection	- improved computational efficiency and accuracy - reduced time complexity	- lack of comparison of the performance (compared only with the SVR)	S
Nurellari et al. [13]	2021	Designing the optimal trajectories for mobile sensors for intrusion detection	- reduced energy consumption - maximised coverage area	- not fully distributed wireless mobile sensor network	M
Saraereh et al. [14]	2021	A method for secondary implementation of mobile sensors using a set-based maximum flow approach	- improved efficiency of computation time	- consuming energy faster	M
Amutha et al. [15]	2021	A distributed border surveillance system in different environments using WSN	- operates in both rectangular and circular regions - operates under both shadowed and non-shadowed environmental conditions	- limited number of sensing models	S
Sharma et al. [16]	2021	Presenting an autonomous surveillance system which is capable of detecting weapons, human infiltrations, weapons, and unmanned aerial vehicles (UAVs)	- capable of detecting the location of the intruder - experimental setup with microcontroller boards	- lack of comparison of the performance	S
Singh et al. [17]	2022	A smart border surveillance system that can identify the intrusion and subsequently send alert messages in the presence of an intruder	- experimental setup with microcontroller boards - has the capability to distinguish between persons and animals	- lack of capability to track the intruder	S
Rajan et al. [18]	2023	Presenting an intrusion detection system for armed attackers	- efficient energy cost	- lack of comparison of the performance	M and S
Mahjoub et al. [19]	2023	Designing WSN using mobile robots for intruder detection and border surveillance	- autonomous travel capability of the sensors	- multiple swarms cannot collaborate to achieve common goals	M

in the approach. For instance, the search for weaknesses in the barrier does not account for the actual intrusions in the environment, and nodes with increased intruder perception consume energy at a faster rate. Moreover, Amutha et al. [15] proposed a distributed border surveillance system in a WSN that uses a log-normal shadowing approach for both circular and rectangular regions. The performance of the system is assessed by analysing the number of barriers obtained to track the region of interest. The influence of varied system and network parameters on the number of barriers is assessed. According to the results obtained, the number of barriers increases in correlation with the number of sensors and sensing range. The proposed surveillance system with the log-normal shadowing approach outperforms the surveillance system with the binary sensing approach. Furthermore, Sharma et al. [16] presented a surveillance system designed for volatile border areas vulnerable to terrorist intrusions. The system is capable of detecting weapons, human infiltrations, weapons, and unmanned aerial vehicles (UAVs) using machine learning algorithms and it determines the location of the intruder with a Global Positioning System (GPS) module. This artificial intelligence-embedded autonomous surveillance system enhances its capabilities, making border surveillance more effective and efficient. It reduces the need for physical presence and surveillance by troops along the fence, minimizing risks to life and allowing existing surveillance equipment to be utilized for other crucial tasks. In a different study, Singh et al. [17] represented a smart border surveillance system utilizing wireless sensors to detect and alert intrusions of intruders. A prototype is developed by integrating Raspberry Pi boards with camera, ultrasonic, and infrared sensors. The prototype communicates via Zigbee serial communication, enabling video streams to be forwarded to a control station for further analysis or action. The developed system has the capability to distinguish between persons and animals. Rajan et al. [18] introduced a WSN model that combines mobile and stationary sensor nodes to enhance intrusion detection against armed attackers. To address the issue of armed intruders, a collaborative approach called the vehicle partnership sensing network is presented, where mobile and static sensor nodes work together for intrusion detection. Furthermore, a system for identifying intrusions is developed. To efficiently track armed intruders, a decentralized target pursuit method utilizing mobile sensor nodes is proposed. Finally, Mahjoub et al. [19] developed a system with mobile robots that mimics natural swarms and it is capable of monitoring borders and detecting intrusions. The robots are equipped with simple sensors such as ultrasonic, camera, temperature, and humidity. Each robot can move autonomously in the swarm.

The methods outlined above are valuable for detecting intruders in border regions. However, they exhibit notable drawbacks such as high computational costs and time

complexity. Due to the extensive data generated by WSNs, a significant amount of time is required for processing and analysis. Time is vital in scenarios like preventing intrusions in border regions. So, even a minor delay in seconds could lead to catastrophic outcomes. Therefore, rapid detection of any intrusion along borders and in restricted areas is crucial. This challenge can be addressed by implementing machine learning approaches known for their computational efficiency [20].

In this study, an effective machine learning model has been proposed to predict the number of k -barriers for rapid and robust intrusion detection and prevention for a rectangular area using features of the WSN. The proposed model integrates the CatBoost algorithm with PSO. While the CatBoost is utilized for predictive modelling tasks, the PSO optimises its parameters to enhance prediction performance. The four features such as network area, transmission range, sensing range, and number of sensor nodes are used for the prediction, which are obtained through the Monte-Carlo simulation process. The performance evaluation of the proposed CatBoost-PSO model has been analysed by using four different metrics such as MAE, MSE, RMSE, and R^2 . The major contributions obtained in this paper are as follows:

- An efficient and robust CatBoost model which is optimised with metaheuristic PSO algorithm has been employed for the k -barriers estimation in wireless systems. This provides a more finely tuned model that demonstrates improved performance in terms of efficiency, accuracy, and adaptability to the specific challenges of intrusion detection in WSNs.
- The proposed model has been compared with state-of-the-art models such as LightGBM, XGBoost, RF, and DT to ensure the reliability of the model.
- The proposed model has been compared with other existing methods in the literature which are implemented for the same dataset to demonstrate the superiority and effectiveness of the model. It has shown better results than existing approaches in all metrics.
- The feature importance of input parameters such as network area, number of sensors, sensing range, and transmission range has been analyzed, with the analysis identifying the network area and transmission range as the two most significant factors in predicting the number of k -barriers. Additionally, timing analyses of the models have been conducted, and the proposed model demonstrates the best prediction time, making it suitable for real-time systems.

Further, the rest of this paper is structured as follows: Sect. 2 presents concise details regarding the communication protocols employed in WSNs. Section 3 provides an overview of the dataset used and the machine learning methods

employed. In Sect. 4, a comprehensive analysis and discussion of the experimental findings are presented. Lastly, Sect. 5 outlines the conclusion of the study.

2 Communication protocols of WSNs

WSNs employ various communication protocols to facilitate effective and reliable data transmission among sensor nodes. These protocols are designed to address the challenges of WSNs such as limited energy resources, dynamic network topology, and the need for scalable and self-organizing communication. Here are some commonly used WSN communication protocols [21, 22]:

- **Bluetooth:** It, specified by the Bluetooth 5.3 standard, is recognized for its low power consumption at 100 mW. With a communication range spanning from 1 to 10 m, Bluetooth operates in the 2.4 GHz frequency band. Employing a point-to-point topology facilitates direct and efficient communication between paired devices.
- **Zigbee:** It, following the IEEE 802.15.4 standard, features low power consumption at 36.9 mW, providing an energy-efficient solution. It operates in the 2.4 GHz frequency band, offering a communication range of 1 to 75 m. With a mesh topology, Zigbee enables interconnected networks, enhancing reliability and flexibility for diverse applications.
- **NFC:** It, following the ISO/IEC 18000–3 standard, operates at a close range of approximately 10 cm. With a frequency of 13.56 MHz, it facilitates short-range wireless communication.
- **Wi-Fi:** It, adhering to various IEEE standards such as 802.11a, b, g, and n, is renowned for its high power consumption at 835 mW. Operating in the 2.4 GHz and 5 GHz frequency bands, it provides a broad communication range of up to 100 m. With a star topology, Wi-Fi networks are structured around a central hub, enabling multiple devices to connect to a central access point for efficient data exchange.
- **Sigfox:** It, operating under the Sigfox standard, is distinguished by its low power consumption, making it an energy-efficient option for different applications. With an impressive communication range of up to 100 kms, it is well-suited for long-range deployments. Employing a star topology, Sigfox networks are organized around a central base station, simplifying the network structure, and enhancing scalability. It operates in the frequency range of 862 to 928 MHz.
- **Cellular Network:** It, operating on the 4G standard, exhibits a communication range spanning from 25 to 35

kms, with frequencies typically ranging from 600 MHz to 2.5 GHz. This technology enables high-speed wireless communication over vast geographical areas.

Designing WSNs necessitates careful consideration of critical factors, owing to challenges arising from environmental characteristics and network application requirements. Addressing issues such as limited energy capacity, fault tolerance, scalability, hardware constraints, latency, range, and production costs is crucial in formulating effective routing protocols for WSNs. It's essential to adapt protocols to specific WSN deployments and use cases, as different protocols may be more suitable for varying scenarios [23]. The proposed algorithm processes the network area, sensing range, transmission range, and number of sensor nodes features to predict the number of k-barriers for intruder detection. Once the necessary features mentioned above are fulfilled, the algorithm can function independently of the protocol. The protocols modify the values of certain features, such as sensing and transmission ranges. Therefore, the generalizability of the proposed method has been demonstrated, regardless of environmental considerations.

3 Materials and methods

3.1 Wireless sensor network dataset

The dataset [24] used in this study was synthetically generated through Monte-Carlo simulations using the NS-2.35 network simulator which is a specific version of the Network Simulator 2 (NS-2) software. The dataset involves five columns which are network area, sensing range, transmission range, number of sensor nodes, and number of barriers. The initial four columns represent the features, while the last column corresponds to the target variable to be predicted.

The features in WSN can be classified into two categories: real and synthetic, based on the data acquisition process. Real data is collected through direct measurements using sensors, but generating real data can be costly and labour-intensive. On the other hand, synthetic data can be generated using mathematical rules, statistical models, and simulations. Compared to real data, acquiring synthetic data is more efficient and cost-effective. Therefore, the dataset has been synthetically generated. Furthermore, the number of barriers has been chosen for the target variable because predicting this variable in WSNs is crucial for optimizing network performance, resource allocation, energy efficiency, and ensuring security and reliability in various deployment scenarios [25].

Table 2 Simulation parameters used in dataset generation

Parameter name	Value
Network area	100×50 m ² to 250×200 m ²
Sensing range	15 to 40 m
Transmission range	30 to 80 m
Number of sensor nodes	100 to 400
Network region	Rectangular region
Deployment type of sensor	Uniform distribution
Sensing model	Binary sensing model
Simulator	NS-2.35

Table 3 Overview of key statistics of each feature

Feature name	Minimum value	Maximum value	Mean value
Network area	5000	50,000	24,375
Sensing range	15	40	27.5
Transmission range	30	80	55
Number of sensor nodes	100	400	250

In this process of generating a dataset, a finite set of sensors is randomly and uniformly placed within a rectangular region. Each sensor is considered to be identical in terms of sensing, transmission, and computational capabilities. The network region dimensions are adjusted, ranging from 100×50 m² to 250×200 m². If two sensors within the deployed WSN meet the condition $R_{tx} > 2R_s$, indicating that the transmission range (R_{tx}) is greater than twice the sensing range (R_s), they can establish communication. In this context, the binary sensing model which is a widely used sensing range model has been adopted to evaluate WSN performance. According to this sensing model, a random sensor can identify a target with a probability of one if the target

falls within its sensing range. Alternatively, if the target is outside the sensing range, the probability of target detection will be zero [25]. The simulation parameters used for dataset generation are given in Table 2.

The dataset consists of 183 examples and an overview of the key statistics of the dataset is provided in Table 3, involving minimum value, maximum value, and mean value. The mean values of the features which are network area, sensing range, transmission range, and number of sensors are 24,375, 27.5, 55, and 250, respectively. The mean value of the target variable is 94.0714.

The distributions of the features can affect the performance of the machine learning models. Having features with smoother distributions contributes positively to the performance of the models. Figure 2 illustrates the histogram of each feature for the dataset used in this experiment. The network area consists of 7 distinct values ranging from 5000 to 50,000. In the data, each value is repeated 26 times. The sensing range, transmission range and number of sensor nodes involve 26 different values ranging from 15 to 40, 30 to 80, and 100 to 400, respectively. Each value is repeated 7 times. Finally, the target variable consists of 122 different values ranging from 12 to 320. Each value is repeated between 1 to 5 times. In conclusion, there is no need for distribution adjustment since the distributions of the features are regular (not skewed to the right or left).

Identifying and addressing outliers in the data is crucial for regression tasks to ensure that the model is not disproportionately influenced by extreme values, which could result in inaccurate predictions or biased parameter estimates [26]. Figure 3 depicts boxplots for each feature, providing insights into the presence of outliers within the dataset. As seen clearly, there is no need for cleaning outliers as there are none present in the features.

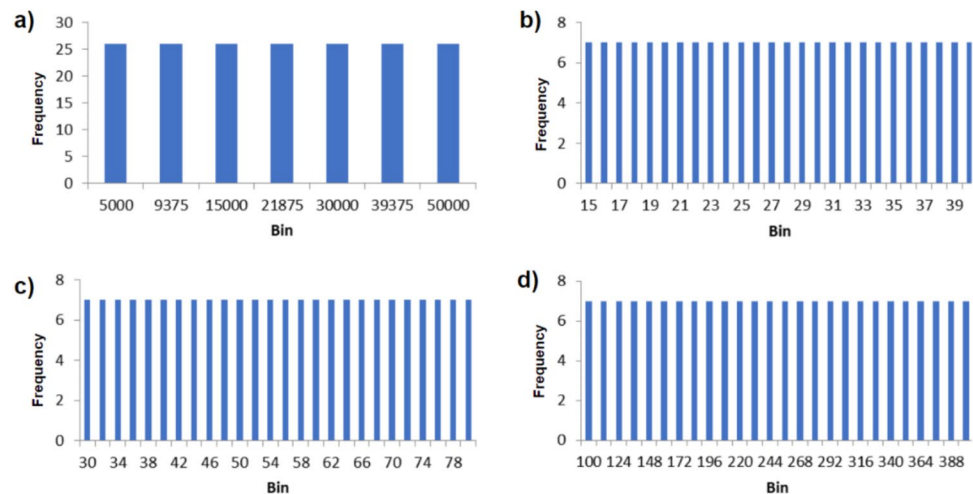
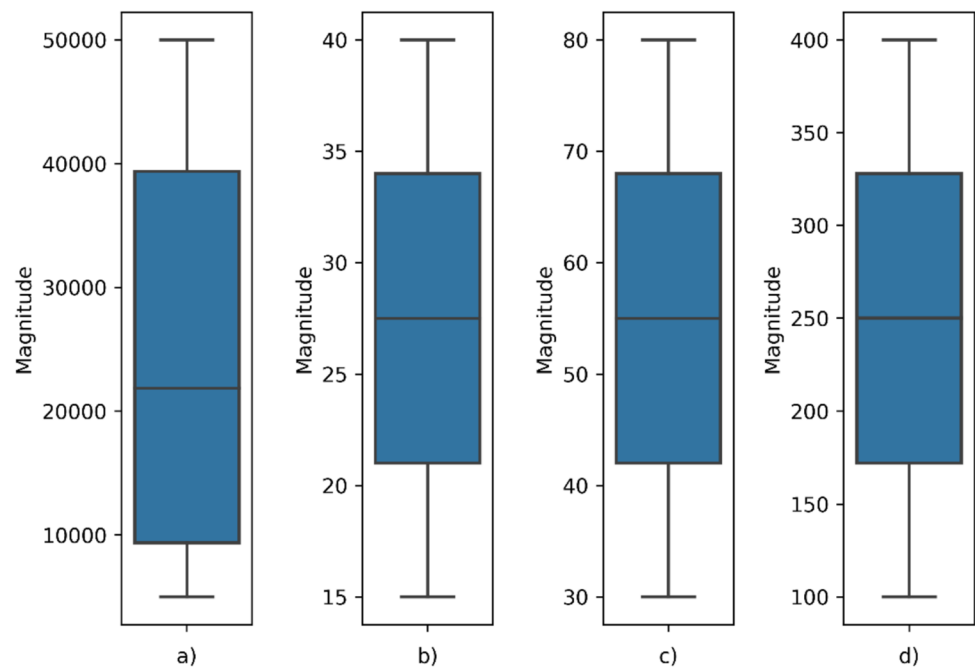
Fig. 2 Histogram of the features; (a) network area, (b) sensing range, (c) transmission range, and (d) number of sensor nodes

Fig. 3 Boxplots of features; (a) network area, (b) sensing range, (c) transmission range, and d) number of sensor nodes



3.2 Machine learning methods

3.2.1 Decision tree algorithm

Decision Tree (DT) is a widely used supervised machine learning algorithm for regression and classification processes. A decision tree employed for classification is referred to as a classification tree, whereas when it is employed for regression, it is referred to as a regression tree. It is based on the binary tree which is one of the popular data structures used in computer science. The aim of the DT is to build a model that estimates the target output by learning simple decision rules from data features, achieved by dividing a complex problem into smaller ones.

The process of creating a DT employs a multistage or hierarchical decision-making scheme. The structure of DT consists of three parts, arranged from top to bottom: a root node, a series of internal nodes, and terminal nodes which are also called leaf nodes. The processing involves traversing down the tree until the leaf is reached.

The DT provides several advantages such as reduced data preparation, interpretability, and efficacy in data exploration [27, 28]. Nevertheless, it also comes with drawbacks such as overfitting, and instability. The tree's structure can be notably influenced by minor changes in the data due to its instability, while increasing its depth may lead to overfitting [29]. To mitigate overfitting, applying pruning after the construction of a tree or limiting the tree's maximum depth can be effective.

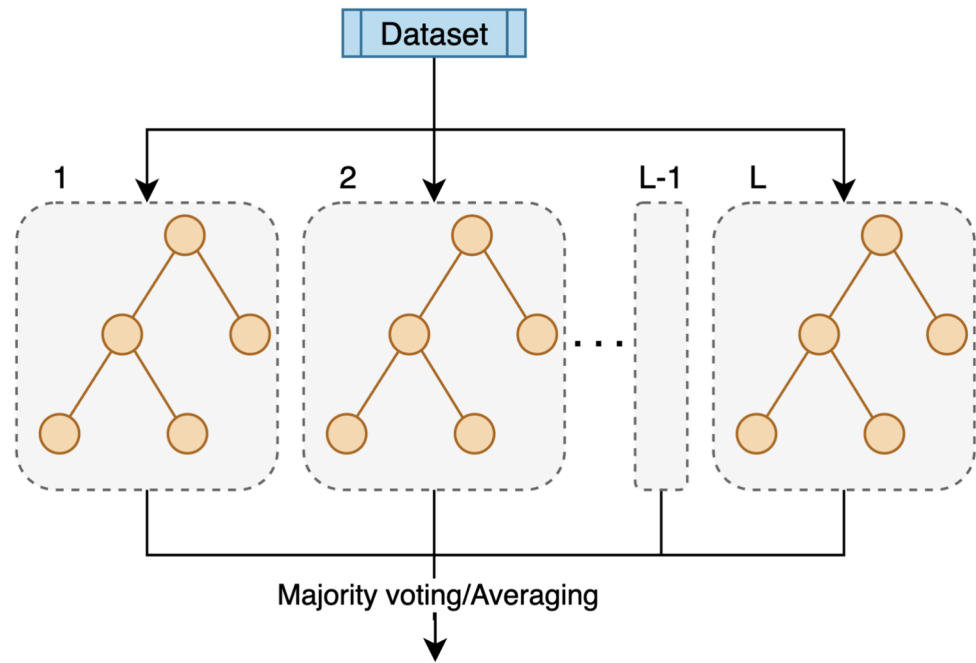
3.2.1.1 Random forest algorithm Random Forest (RF), proposed by Breiman (2001) [29], is an ensemble learning algorithm based on decision trees. It is used for both classification and regression problems. The RF gathers information by randomly selecting features from each decision tree in the forest. It then employs a majority voting approach for classification or an averaging approach for regression problems to reach the final prediction.

The RF exhibits robustness against overfitting, a common concern in machine learning. Additionally, RF demonstrates a noteworthy capability in handling large datasets characterized by high dimensionality, making it a suitable choice for complex and extensive data scenarios. Besides the advantages of RF, there are also disadvantages. One such limitation is its potential underperformance when encountered with imbalanced datasets [30].

Figure 4 illustrates the operation of the RF as an ensemble model with L decision trees. Each tree independently processes inputs (x_i) and produces individual output estimations (P_i^*) for $i = 1, \dots, L$. These estimations from the decision trees are then aggregated to generate the final prediction. In classification, the final prediction (P_{final}) is determined by the majority of trees, while in regression, it is computed as the average of the decision trees' outputs by using Eq. 1.

$$P_{final} = \frac{1}{L} \sum_{i=1}^L P_i^* \quad (1)$$

Fig. 4 Structure of RF algorithm

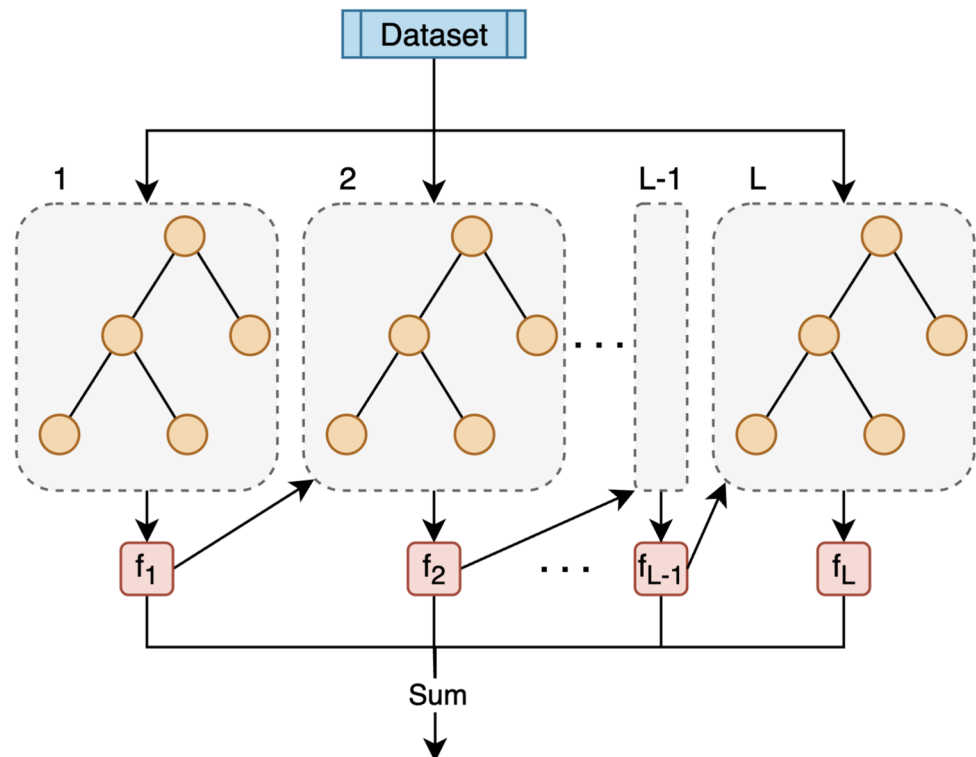


The RF's strength lies in its ability to create diverse decision trees within the forest. This is achieved by utilizing different subsets of the training dataset through the bagging process. By allowing each decision tree to be trained on distinct sets of data, the ensemble model can

effectively prevent overfitting and enhance its overall performance and generalization capability.

3.2.1.2 Extreme gradient boosting algorithm Extreme Gradient Boosting (XGBoost), proposed by Guestrin and Chen (2016) [31], is a supervised machine learning tech-

Fig. 5 Structure of XGBoost algorithm



nique based on gradient boosting decision tree. As depicted in Fig. 5, it represents a boosting model that utilizes a sequential ensemble technique, combining multiple basic regression trees in a series. It involves sequentially adding and training new decision trees to adjust the residuals from previous iterations [32]. The estimation values of all these decision trees are then accumulated to derive the ultimate estimation outcome. Meanwhile, the XGBoost enhances estimation performance by minimizing bias in the model and substantially lowering the risk of model overfitting [33].

In contrast to the conventional gradient boosting decision tree algorithm, the XGBoost alters the objective function as follows:

$$F_{Loss}^{(k)} = \sum_{i=1}^K \left[g_i \cdot \delta_k(x_i) + \frac{1}{2} \cdot h_i \cdot \delta_k^2(x_i) \right] + \Omega(\delta_k) \quad (2)$$

$$g_i = \partial_{\hat{y}^{(k-1)}} \cdot l(y_i, \hat{y}_i^{(k-1)}), h_i = \partial_{\hat{y}^{(k-1)}}^2 \cdot l(y_i, \hat{y}_i^{(k-1)}) \quad (3)$$

$$\Omega(\delta_k) = \alpha \cdot K + \frac{1}{2} \cdot \lambda \cdot \sum_{n=1}^K \omega_n^2 \quad (4)$$

where;

- $\delta_k(x_i)$ indicates the score of the i^{th} sample in the k^{th} tree,
- $\Omega(\delta_k)$ indicates the regular term,
- g_i is the first partial derivative of the loss function,
- h_i is the second partial derivative of the loss function,
- y_i indicates the observed value,
- $\hat{y}_i$ indicates the predicted value,
- l indicates the tree quantity,
- α indicates the complexity of the terminal nodes,
- λ indicates the scaler factor,
- K indicates the number of the leaves,
- ω_n indicates the weight of the n^{th} terminal node in the tree.

The objective function, after eliminating the constant term, can be obtained as follows:

$$F_{Loss} = -\frac{1}{2} \cdot \sum_{j=1}^K C + \alpha \cdot K \quad (5)$$

where C indicates the contribution of each leaf node to the current model loss function and is obtained as follows:

$$C = \frac{G_n^2}{H_n + \lambda} \quad (6)$$

The XGBoost traverses all split leaf nodes by employing the greedy algorithm. If the gain of the target after the split is below the predefined threshold, the split can be ignored.

The XGBoost does not utilize entropy or information gain for decision tree splits; instead, it employs the following gain metric:

$$Gain = \frac{1}{2} \cdot (S_{Left} + S_{Right} - S_{Both}) - \alpha \quad (7)$$

where S_{Left} indicates the score of the left subtree, S_{Right} indicates the score of the right subtree, and S_{Both} indicates the score when it is not split. These scores are obtained as follows:

$$S_{Left} = \frac{G_L^2}{H_L + \lambda} \quad (8)$$

$$S_{Right} = \frac{G_R^2}{H_R + \lambda} \quad (9)$$

$$S_{Both} = \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \quad (10)$$

3.2.1.3 Light gradient boosting machine algorithm Light Gradient Boosting Machine (LightGBM), proposed by Ke et al. (2017) [34], is an efficient and open-source gradient boosting decision tree based algorithm. It is used for regression, sorting and classification problems. The primary distinction from the XGBoost, the LightGBM employs a histogram based algorithm, resulting in an accelerated training process and decreased computational complexity [35, 36]. Meanwhile, it uses a leaf-wise growth strategy with a pre-defined maximum depth parameter. By using this strategy, the LightGBM avoids the simultaneous splitting of leaves on the same layer, as seen in the level-wise growth strategy which is used by XGBoost. This can enhance accuracy by reducing training errors. However, this improvement in accuracy comes with the potential downside of increasing the risk of overfitting. This trade-off between accuracy improvement and overfitting risk can be mitigated by setting an appropriate maximum depth limit. Therefore, setting a maximum depth limit is crucial for developing a robust and generalizable model.

3.2.1.4 Categorical boosting algorithm Categorical Boosting (CatBoost), proposed by Prokhorenkova et al. (2018) [37], is a gradient boosting decision tree algorithm. It is developed to manage categorical features more efficiently. The CatBoost utilizes enhanced and revised target based statistics (Greedy TBS). Within this procedure, categorical features can be replaced with their corresponding average label values. These average label values are then served as the criteria for node splitting during the construction of decision trees. This innovative approach effectively minimizes

overfitting and accelerates the estimation of the model. The Greedy TBS can be obtained as follows [36]:

$$\frac{\sum_{j=1}^p [x_{j,t} = x_{i,t}] \cdot Y_i}{\sum_{j=1}^p [x_{j,t} = x_{i,t}]} \quad (11)$$

In most cases, features involve more information than labels. However, forcefully using the average label value to represent features may cause a conditional shift, which refers to a change in the relationship or distribution between features and labels due to the way in which the average label value is used to represent features [38]. To handle this situation, CatBoost put a prior value using Greedy TBS. Consider a dataset of observations $O = \{X_i, Y_i\}$, $i = 1, \dots, N$, where $X_i = (X_{i,1}, \dots, X_{i,M})$ is a vector including both categorical and numerical features, with M representing the number of features. Y_i , which is the corresponding label for each observation, is c real number.

Initially, the dataset undergoes a random permutation, which can lead to overfitting. For each sample within the

permutation, the mean label value is computed using the samples that share the same category value before the given one to mitigate overfitting. if a permutation is $\sigma = (\sigma_1, \dots, \sigma_N)$, the permuted observation $x_{\sigma_{p,t}}$ is substituted with [36]:

$$x_{\sigma_{p,t}} = \frac{\sum_{j=1}^{p-1} [x_{\sigma_{j,t}} = x_{\sigma_{p,t}}] \cdot Y_{\sigma_j} + c \cdot P}{\sum_{j=1}^{p-1} [x_{\sigma_{j,t}} = x_{\sigma_{p,t}}] + c} \quad (12)$$

where P is a prior value which refers the mean label value in regression, c is the weight of the prior value which is greater than 0. This approach leads to reducing the noise for the low frequency categories [29].

During the construction of a new split node in the current tree, the CatBoost utilizes a greedy technique to explore all possible combinations of different feature types. It dynamically transforms these combined categorical features into numeric features (see Fig. 6). Additionally, ordered boosting is employed by CatBoost to replace the gradient prediction in the traditional boosting algorithm, effectively handling

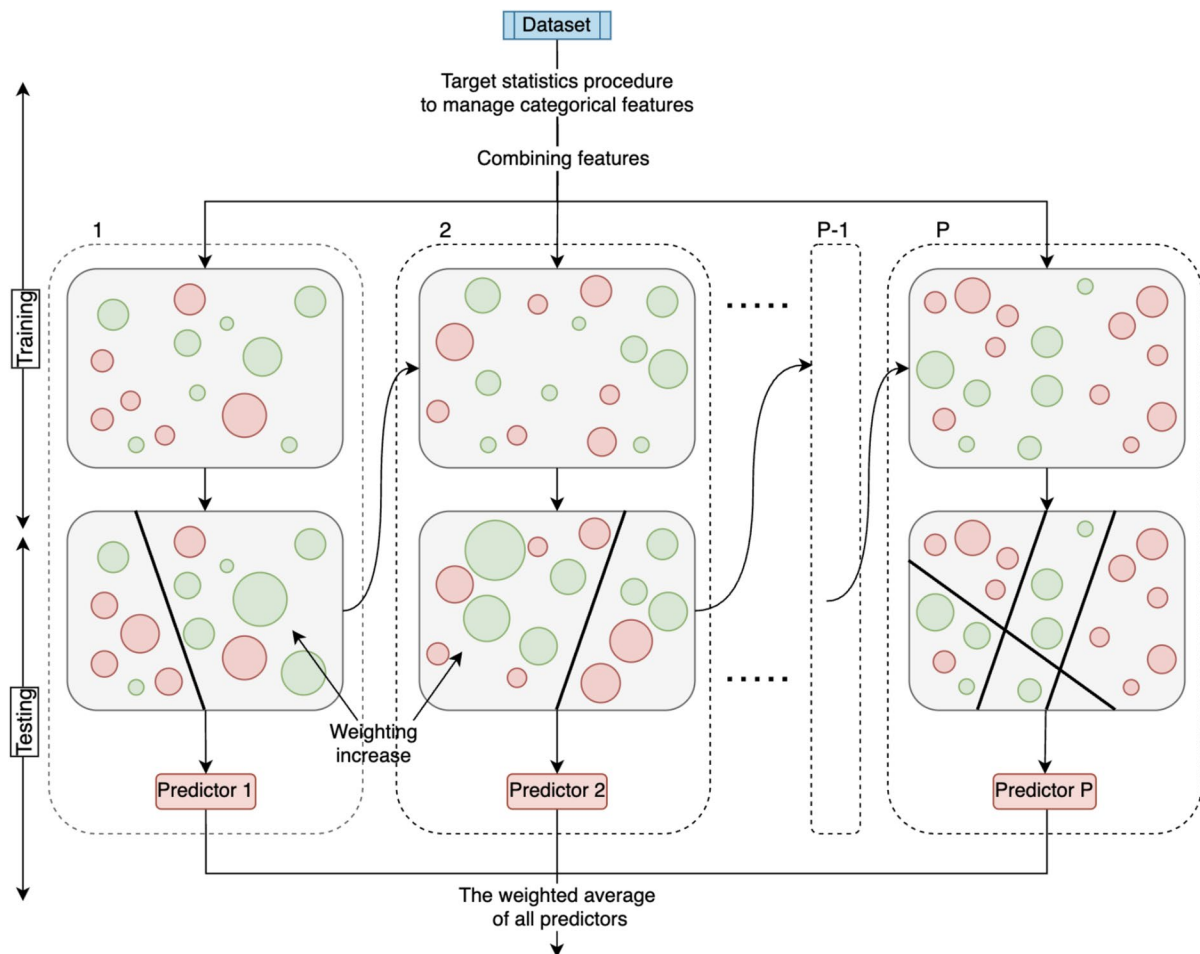


Fig. 6 Structure of CatBoost algorithm

estimation shifts induced by gradient bias and improving the generalization ability of the model.

3.3 Particle swarm optimisation algorithm

Particle Swarm Optimisation (PSO), proposed by Kennedy and Eberhart [39], is a metaheuristic optimisation algorithm that is first inspired by seeking the food behaviour of birds. The PSO possesses several advantages, including simplicity, relatively low computational cost, and effectiveness [40]. Thanks to these advantages, it has gained substantial popularity in the field of computer science in recent times [39, 41–45].

The PSO is initialized with random solutions called particles. Each particle i has a current position and velocity vectors, which are $V_i = [v_i^1, v_i^2, \dots, v_i^D]$ and $X_i = [x_i^1, x_i^2, \dots, x_i^D]$, respectively. The process begins with a randomized V_i and X_i . Then, the particles traverse the multidimensional problem space, remembering their best solution (P_{Best}). The global best solution (G_{Best}) is also tracked. During each iteration, the particles adjust their velocities and positions in every dimension towards P_{Best} and G_{Best} locations with the Eq. 13 and Eq. 14 [45]. The PSO efficiently converges to the optimal solution.

$$v_i^d(t+1) = v_i^d(t) + c_1 \cdot \varepsilon_1 \cdot (P_{Best_i}^d(t) - x_i^d(t)) + c_2 \cdot \varepsilon_2 \cdot (G_{Best}^d(t) - x_i^d(t)) \quad (13)$$

$$x_i^d(t+1) = x_i^d(t) + v_i^d(t+1) \quad (14)$$

where c_1 and c_2 are two positive acceleration constants. ε_1 and ε_2 are two uniform random values generated between $[0,1]$ intervals in each iteration.

4 Results and Discussion

In this study, the CatBoost algorithm, which is optimised with a metaheuristic PSO approach, was implemented to forecast the optimal number of barriers needed for intrusion detection and prevention. The performance of the model was compared with the other state-of-the-art models such as LightGBM, XGBoost, RF, and DT. These models were also optimised with PSO to ensure an unbiased comparison. The feature importance of the input parameters was analysed for all models. Moreover, the performance of the model was compared with the other existing methods in the literature that use the same dataset. The flowchart representing the detailed methodology can be seen in Fig. 7.

The dataset was divided into two subsets: a training set comprising 75% of the total data, and a test set comprising the remaining 25%. To assess the effectiveness of each model, the training process was employed with fivefold cross-validation. The experiments were carried out in two phases. In the first phase, the algorithms were employed using a grid search configuration with limited configurations. In the second phase, hyperparameter optimisation

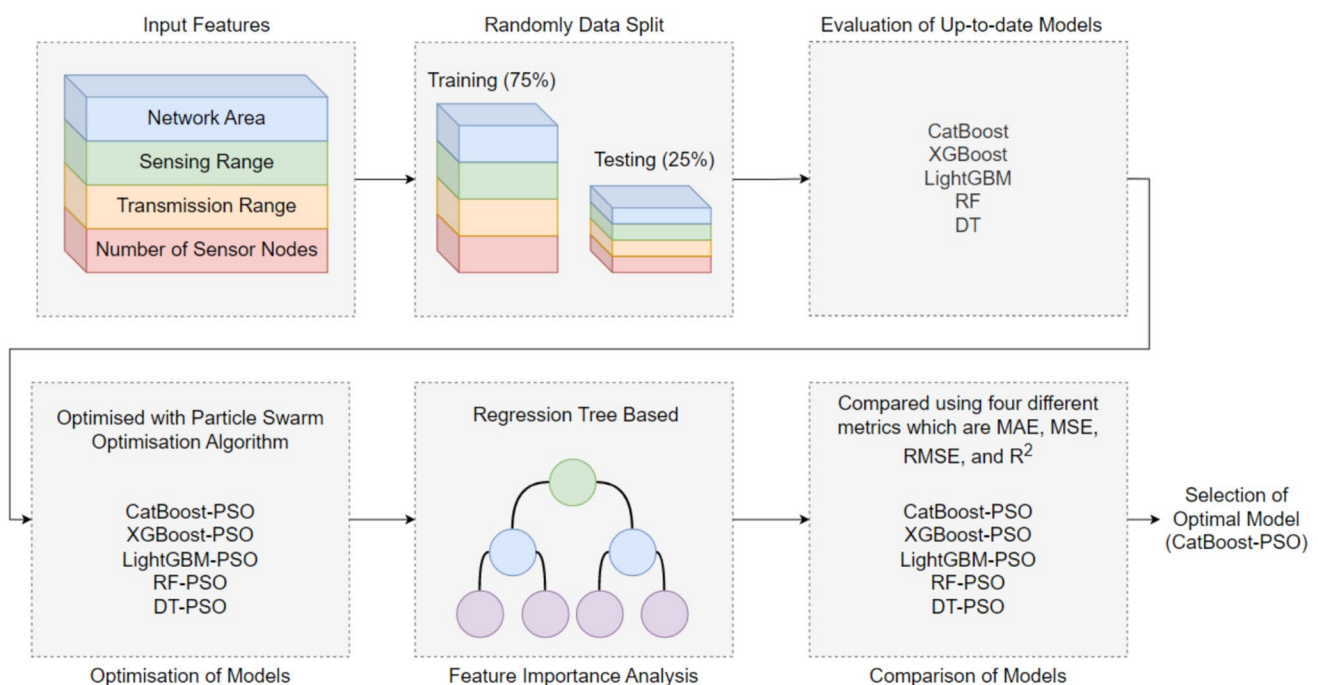


Fig. 7 Flowchart representing the detailed methodology

was employed using the metaheuristic PSO algorithm to enhance the performance of the model. The robustness of the PSO algorithm was demonstrated through 30 tests with different seeds, using a parameter setting in Table 4.

The pseudo-code of the implemented methodology is as follows:

1. For each model m ($M_m = [DT, RF, LightGBM, XGBoost, CatBoost]$)
2. Repeat until reaches maximum runs (1 to 30)
3. Initialize particle swarm with random velocities ($V_i = [v_{i1}, v_{i2}, \dots, v_{iD}]$)
4. Initialize particle swarm with random positions ($X_i = [x_{i1}, x_{i2}, \dots, x_{iD}]$)
5. Set PSO parameters ($c_1 = 0.5$, $c_2 = 0.3$, $w = 0.9$, population size = 20)
6. Set the maximum and minimum values of each hyperparameter of the model m
7. Repeat until reaches maximum iteration (1 to 50)
8. For each particle i in the swarm
9. Initialize hyperparameters of the model m within given ranges randomly
10. Evaluate fitness value using the objective function (lowest MSE)
11. If fitness value is better than personal best (P_{Best}):
12. Set current fitness values as the P_{Best}
13. Choose particle i with the best fitness value as G_{Best}
14. For each particle i in the swarm
15. Update velocity with the Eq. 13
16. Update position with the Eq. 14
17. Train model m with the optimised hyperparameters on the training set
18. Evaluate the performance of model m on the testing set by MAE, MSE, RMSE, and R^2
19. Store the performance results of model m
20. Compare the stored results and select the optimal model

By providing a structured and readable pseudo-code, the paper aims to facilitate a smoother implementation of the proposed approach and encourages collaboration and further

exploration in the field of intrusion detection in WSNs using machine learning methods optimised with PSO.

4.1 Evaluation metrics

The performances of the models are compared by employing four various evaluation metrics which are the mean absolute error (MAE), the mean squared error (MSE), the root mean squared error (RMSE), and the coefficient of determination (R^2). These metrics are obtained as follows:

$$MAE = \frac{\sum_{i=1}^N |A_i - E_i|}{N} \quad (15)$$

$$MSE = \frac{\sum_{i=1}^N (A_i - E_i)^2}{N} \quad (16)$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (A_i - E_i)^2}{N}} \quad (17)$$

$$R^2 = \frac{\sum_{i=1}^N (A_i - E_i)^2}{\sum_{i=1}^N (A_i - \bar{A})^2} \quad (18)$$

where N is the number of tests, A_i is the actual value, $\bar{A}$ is the average of the actual values, and E_i is the estimated value.

The R^2 value lies within the range of 0 to 1. A score of 1 represents perfect precision, while a score of 0 indicates that the model's predictions are at their worst. The other three metrics which are MAE, MSE, and RMSE are highly correlated with each other [46]. Their values range from 0 to $+\infty$. Lower values of RMSE, MSE, and MAE point out better model performance, while higher values represent better model performance for R^2 .

4.2 Hyperparameter optimisation

The best configuration for the decision tree-based machine learning models has been determined through a hyperparameter optimisation approach. Each model has its own set of internal parameters that need to be fine-tuned in order to achieve its best performance. To accomplish this, the PSO was employed for optimizing these parameters.

In selecting the hyperparameters for the machine learning models employed in this study, careful consideration was given to balancing model complexity, predictive performance, and computational efficiency. For the CatBoost algorithm, specific hyperparameters such as max_depth and iterations were selected to manage effectively the complexity of the tree and to avoid overfitting. Also, learning_rate was

Table 4 Values of the PSO parameters used in the experiment

Parameter	Value
c_1 (cognitive coefficient)	0.5
c_2 (social coefficient)	0.3
w (inertia weight)	0.9
Number of iterations	50
Number of particles	20

adjusted with precision to maintain a stable training process while ensuring optimal convergence speed. Similarly, hyperparameters of XGBoost such as eta and max_depth were carefully chosen to achieve a trade-off between the speed of convergence and the complexity of the model. The hyperparameters of LightGBM such as num_leaves and min_data_in_leaf were tailored to prevent overfitting while keeping the tree structure manageable. The hyperparameters of RF such as n_estimators and max_features were fine-tuned to manage model complexity efficiently. Additionally, hyperparameters of DT were meticulously chosen to regulate tree depth and criteria for node splitting. All selections of hyperparameters were informed by empirical evidence, domain expertise, and extensive experimentation to ensure the optimal performance of the model for the given prediction task. Table 5 presents the hyperparameters of the models and their descriptions, lower–upper bounds, and optimal values obtained after optimisation process.

The R^2 results of the models, obtained through hyperparameter optimisation with 30 independent runs, are presented in Table 6. The performance comparison of the

Table 6 Statistical analysis of R^2 values of the models

Model name	Mean	Standard deviation	Best
CatBoost	0.9997	7.2476e-05	0.9998
XGBoost	0.9983	0.0004	0.9989
LightGBM	0.9826	0.0267	0.9903
RF	0.9871	0.0103	0.9919
DT	0.9781	0.0119	0.9821

Best results are in bold*

models was conducted using tree metrics; the mean of R^2 , the standard deviation of R^2 , and the best of R^2 . These metrics provide insights into the overall performance and variability of the models' results.

As given in Table 6, the highest R^2 value has been achieved by the CatBoost with the best value of 0.9998. Moreover, it demonstrates a mean value of 0.9997 and a standard deviation of 7.2476e-05, highlighting its consistency and performance superiority. On the other hand, the LightGBM presents the largest standard deviation with a

Table 5 Hyperparameters of the models and their descriptions, lower and upper bounds, and optimal values obtained by PSO

Model name	Hyperparameter	Description	Range	Optimal value
CatBoost	max_depth	Maximum depth of a tree	[1, 15]	5
	learning_rate	A parameter that determines the step size at each iteration	[0.01, 0.1]	0.0985
	iterations	Number of trees	[10, 400]	375
	random_strength	The amount of randomness to use for scoring splits	[0, 0.2]	0.0593
	l2_leaf_reg	L2 regularization term for the leaf weights	[1, 10]	1
XGBoost	eta	A parameter that determines the step size at each iteration	[0.1, 0.9]	0.2130
	max_depth	Maximum depth of a tree	[3, 20]	3
	n_estimators	Number of trees	[10, 200]	200
	alpha	L1 regularization term on weights	[0, 0.0001]	0.0001
	lambda	L2 regularization term on weights	[0.5, 1]	0.5
	min_child_weight	Minimum sum of instance weight needed in a child	[1, 6]	1
	subsample	Subsample ratio of training samples	[0.6, 1]	0.6
LightGBM	num_leaves	Maximum tree leaves	[2, 100]	7
	min_data_in_leaf	The minimum number of data points required in a leaf node	[10, 300]	10
	max_depth	Maximum tree depth	[1, 15]	15
	learning_rate	A parameter that determines the step size at each iteration	[0.01, 0.1]	0.0999
RF	n_estimators	Number of trees	[5, 400]	12
	min_samples_split	Minimum number of samples placed in a node before the node is split	[2, 50]	2
	min_samples_leaf	Minimum number of samples allowed in a leaf node	[1, 50]	1
	max_depth	Maximum depth of a tree	[1, 100]	45
	max_features	Maximum number of features considered for splitting a node	[1, 15]	14
DT	min_weight_fraction_leaf	Minimum weighted fraction of the total sum of weights required to be at a leaf node	[0, 0.5]	0.0
	max_features	Maximum number of features considered for splitting a node	[0.4, 1]	0.4
	min_samples_split	Minimum number of samples placed in a node before the node is split	[2, 10]	2
	min_samples_leaf	Minimum number of samples allowed in a leaf node	[1, 20]	1
	max_depth	Maximum depth of a tree	[1, 15]	13

value of 0.0267, indicating that it is slightly more unstable compared to the other models.

The R^2 results are also visualized in Fig. 8, illustrating the distributions of all 30 values of R^2 for each model in detail. In the visualization, the red dashed line indicates the mean value, while the green dashed line indicates the median value. It is observed that in the proposed CatBoost model, these two lines are closest to each other, indicating the robustness of the model [22].

4.3 Performance comparison with other up-to-date models

In Fig. 9, the convergence of the PSO algorithm during the search process is illustrated. The figure depicts the convergence behaviour of the optimisation algorithm. It reveals that during the initial 10 iterations, the algorithm rapidly converges towards local optima. However, its progress gets impeded between iterations 10 and 20, suggesting a potential difficulty in escaping local optima during this period. After this period, the overall convergence speed shows a slight

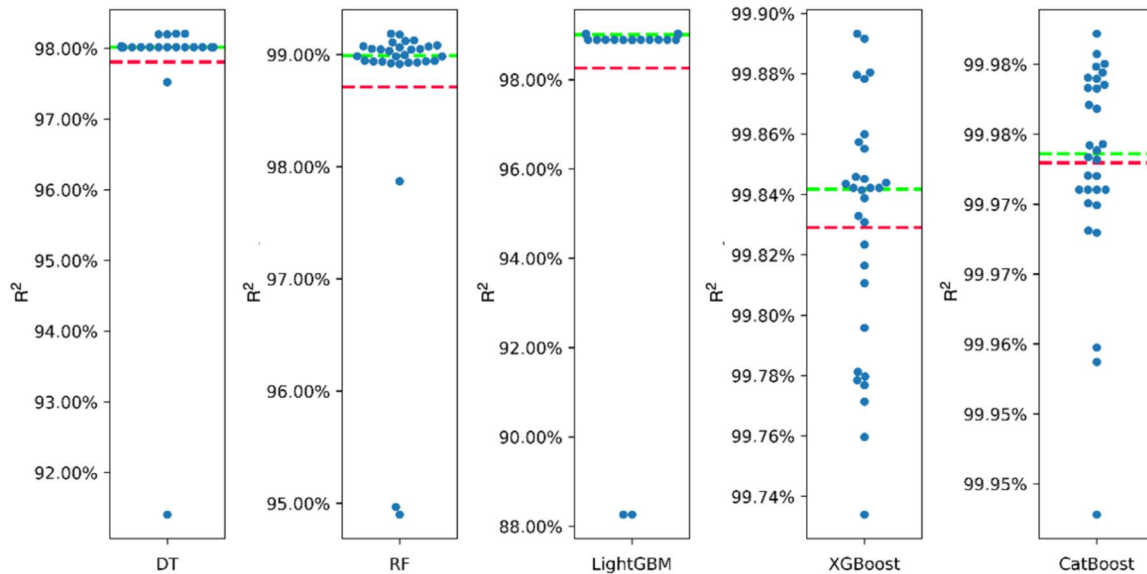
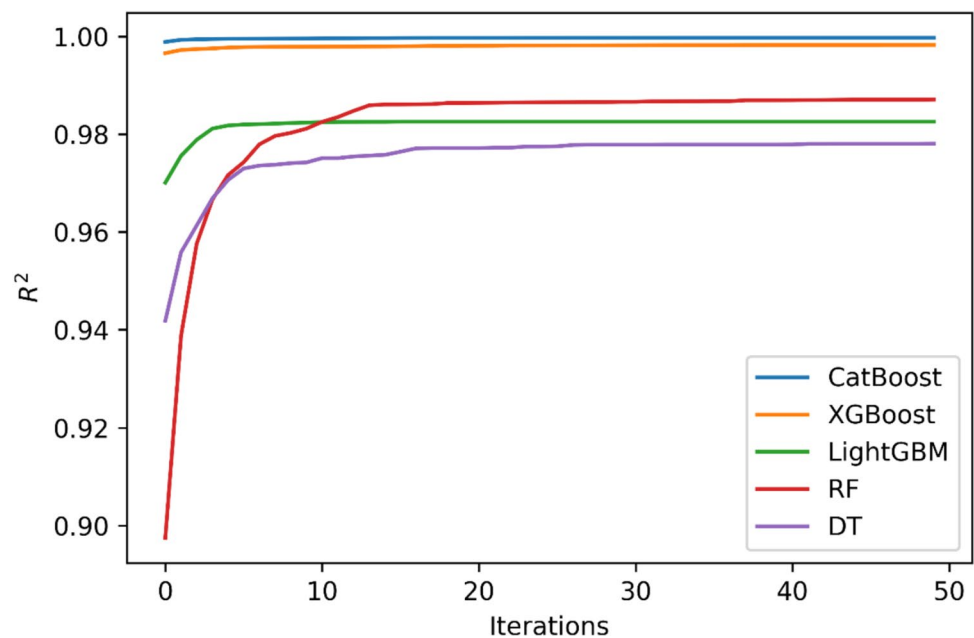


Fig. 8 Distributions of the R^2 values obtained from the models

Fig. 9 Convergence graph of the PSO process



decrease. The mean R^2 values derived from 30 models configured using the PSO algorithm are 0.9997, 0.9983, 0.9826, 0.9871, and 0.9781 for CatBoost, XGBoost, LightGBM, RF, and DT, respectively. Notably, the CatBoost model exhibits the most outstanding performance and the XGBoost model is following it in second place. On the contrary, the DT model shows the worst performance among the five models.

The performance of the CatBoost is compared with other up-to-date models such as XGBoost, LightGBM, RF, and DT by using their best configurations. The comparison results are given in Table 7. In addition to the R^2 value, the other evaluation metrics which are MAE, MSE, and RMSE values are also used to strengthen the comparison. As clearly seen, the CatBoost outperforms the XGBoost, LightGBM,

RF, and DT with values of 0.9998 R^2 , 0.6298 MAE, 0.6018 MSE, and 0.7758 RMSE. The DT shows the worst performance considering all metrics.

Figure 10 depicts the estimated number of barriers for each model alongside the actual number of barriers for all models. The results show that the data are not uniformly distributed and are mostly clustered within the 0–100 range. The CatBoost model demonstrated satisfactory performance in estimating the number of barriers with a narrower error range compared to XGBoost, LightGBM, RF, and DT. The DT model performed poorly with a wider error range. As a result, built-in techniques of the CatBoost like random permutations and bootstrap reduce overfitting, making it advantageous for smaller datasets. Additionally, its ability to handle missing data during both training and prediction eliminates the need for explicit imputation strategies, making it beneficial for datasets with missing values.

Table 7 Performance comparison of the models

Model name	MAE	MSE	RMSE	R^2
CatBoost-PSO	0.6298	0.6018	0.7758	0.9998
XGBoost-PSO	1.4660	3.6055	1.8988	0.9989
LightGBM-PSO	4.2256	32.8267	5.7295	0.9903
RF-PSO	3.9239	27.4825	5.2424	0.9919
DT-PSO	7.1304	60.5652	7.7824	0.9821

Best results are in bold*

4.4 Analysis of time complexity

Time complexity is a crucial metric in the analysis and evaluation of algorithms, offering insights into their efficiency and scalability. Expressed using Big-O notation, time complexity characterizes the upper bound of an algorithm's runtime, representing the worst-case scenario. Common

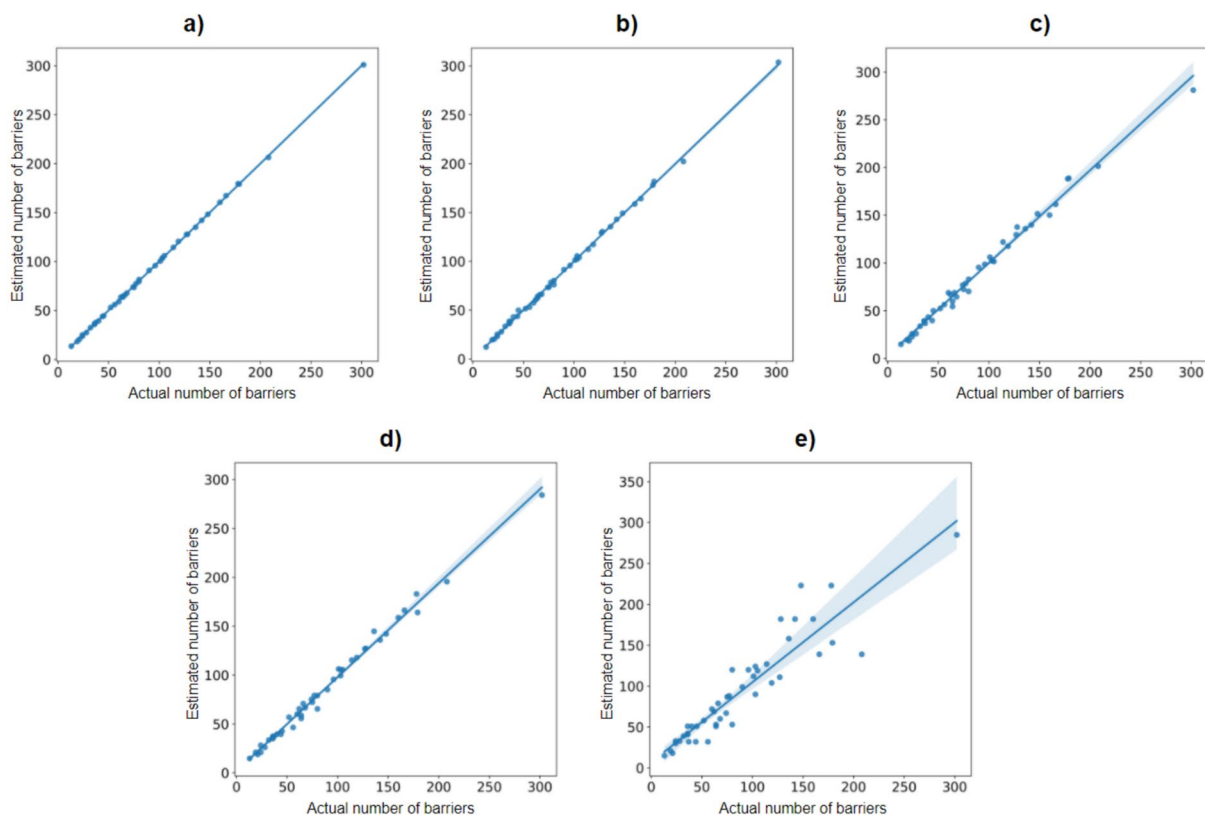


Fig. 10 Regression lines of the models; (a) CatBoost, (b) XGBoost, (c) LightGBM, (d) RF, and (e) DT

time complexities involve constant ($O(1)$), linear ($O(n)$), logarithmic ($O(\log n)$), polynomial ($O(n^a)$), quasilinear ($O(n \log n)$), factorial ($O(n!)$), and exponential ($O(a^n)$). A lower time complexity generally implies faster algorithmic performance, making it a key consideration in algorithm design and selection [47].

The training time, prediction time and space complexities of the models used in this study are given in Table 8 [31, 48]. The parameters are defined as follows:

- m is the number of features,
- n is the number of data points,
- n_{tree} refers to the number of individual decision trees,
- n_{boost} represents the number of boosting rounds,
- n_{ensemble} refers to the number of leaves in the ensemble,
- K is the number of unique categories across all categorical features,
- $n_{\text{categories}}$ is the total number of unique categories across all features (both categorical and numerical)

While LightGBM, XGBoost, and CatBoost are based on gradient boosting trees and share a similar overall time complexity, there are some nuances to consider. LightGBM utilizes leaf-wise splitting and histogram-based decision tree learning, which can lead to faster training compared to XGBoost's level-wise approach and exact gradient calculations. On the other hand, CatBoost excels in handling categorical data and can be fast in specific scenarios, but its overall training speed might be much slower than that of LightGBM and XGBoost.

Table 9 presents the training and prediction times of each individual model. To assure the reliability of the computational results, every model underwent 10 runs on two different devices (D1 and D2), each with distinct specifications. The average run times have been calculated separately for each device. D1 is equipped with an Intel (R) Core (TM) i7-8700 CPU (6 Cores) running at 3.20 GHz, 16 GB RAM, and Windows 10 Pro 64-bit. D2, on the other hand, features an Intel (R) Core (TM) i9-12,900 CPU (16 Cores) running at 2.4 GHz, 64 GB RAM, and Microsoft Windows 11 Pro 64-bit.

Table 8 Training time, prediction time, and space complexities of the models

Model name	Training time complexity	Prediction time complexity	Space complexity
DT	$O(n*m*\log(n))$	$O(\log(n))$	$O(n*m)$
RF	$O(n*m*\log(n)*n_{\text{tree}})$	$O(n_{\text{tree}}*\log(n))$	$O(n*m*n_{\text{tree}})$
LightGBM	$O(n*m*\log(M)*n_{\text{boost}})$	$O(n_{\text{ensemble}})$	$O(n*m)$
XGBoost	$O(n*m*\log(M)*n_{\text{boost}})$	$O(n_{\text{ensemble}})$	$O(n*m)$
CatBoost	$O(n*m*\log(M)*n_{\text{boost}})$	$O(n_{\text{ensemble}})$	$O(n*m + k*n_{\text{categories}})$

Table 9 Comparison of the computing times of the models

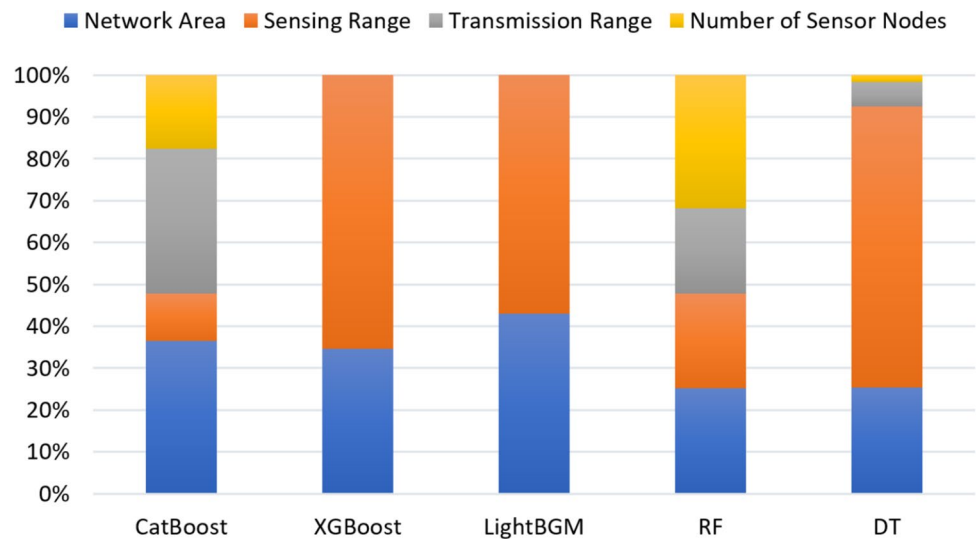
Model name	Training time (ms)		Prediction time (ms)	
	D1	D2	D1	D2
DT-PSO	1.5930	1.2701	0.7084	0.4787
RF-PSO	72.5962	45.7289	3.6933	1.5708
LightGBM-PSO	8.8765	6.2572	2.1063	1.5566
XGBoost-PSO	67.4722	33.3975	2.0150	1.6232
CatBoost-PSO	365.0140	116.0589	0.7009	0.4326

Best results are in bold

The computational results reveal that all suggested machine learning models exhibit the capacity to predict the number of barriers within milliseconds. This efficiency highlights the practicality of employing these models for real-time predictions for intruder detection [49, 50]. Although the proposed CatBoost-PSO model has demonstrated the worst training time performance, it shows the best prediction time performance. To assess the speed of the system, it is essential to consider the prediction time rather than the training time. This is because the model will be trained once before it starts working live. Therefore, the proposed model is the best in terms of real-time operation.

4.5 Analysis of feature importance

The prominence of each feature has been evaluated for all models. This evaluation process has been conducted by using the Mean Decrease Impurity (MDI) approach. It is commonly used feature importance algorithm used for calculating the overall importance of each feature in decision tree-based ensemble methods [51]. MDI determines the significance of each feature by assessing its impact on reducing impurity within the tree nodes during model training. Features that contribute to the most significant decrease in impurity are regarded as more important. Since all models used in this study are decision tree-based, the MDI algorithm is preferred for feature importance analysis. The bars depicted in Fig. 11 represent the relative feature importance score of each feature. As clearly seen, in the estimation of the number of barriers for intrusion detection and prevention using the CatBoost, the network area and the transmission

Fig. 11 Feature importance of the input parameters

range appear as the most relevant features. However, for the XGBoost and LightGBM, only the network area and sensing range demonstrate significant effects, while the other features appear to have no influence on the estimation. Interestingly, all four features exhibit almost equal importance in the RF model, showing similar contributions to the estimation. As for the DT model, the sensing range stands out as the most effective feature, followed by the network area.

4.6 Performance comparison with other existing methods in the literature

In this study, a comprehensive performance comparison of the proposed method with other existing methods in the literature was conducted, all of which have been previously employed over the same dataset. Due to the dataset being relatively new, with its release date in January 2022 [24], there is a limited number of studies available so far. In each study, multiple machine learning methods were employed, and their results were presented. These outcomes are provided in Table 10.

As clearly seen in Table 10, the proposed model, which is CatBoost optimised with PSO, outperforms other existing methods in estimating the number of barriers for intrusion detection and prevention with MAE value of 0.6298, MSE value of 0.6018, RMSE value of 0.7758, and R^2 value of 0.9998. On the contrary, the ANN shows the worst performance with R^2 value of 0.38. Considering the MSE and RMSE metrics, it can be observed that the GPR model exhibits the poorest performance among all the models listed in the table with MSE value of 0.4074.70, RMSE value of 63.83. Compared to the MAE, MSE and RMSE metrics, it can be argued that the ANN model is

Table 10 Performance comparison of the proposed model with other existing methods

Model name	MAE	MSE	RMSE	R^2
CatBoost-PSO (This paper)	0.6298	0.6018	0.7758	0.9998
LT-FS-ID [25]	-	41.87	6.47	0.98
ANN [25]	-	2150.20	46.37	0.38
GRNN [25]	-	3312.00	57.56	0.96
RF [25]	-	1033.60	32.15	0.99
GPR [25]	-	4074.70	63.83	0.94
FANN [2]	-	-	48.36	0.79
IT3FLS [52]	-	-	5.36	0.997
P ² CA-GAM-ID [53]	-	-	15.22	0.97
ENFS-Uni0-reg [4]	-	-	11.16	-

Best results are in bold*

relatively worse due to the R^2 metric being more informative for performance evaluation [46, 54].

CatBoost utilizes an optimised ordered boosting technique, enhancing the training process and enabling more effective learning from data. This strategy contributes to improved performance, particularly in regression tasks where accurately capturing relationships between features is crucial. Also, this technique naturally handles categorical data, making it convenient for datasets with a mix of categorical and numerical features [55, 56]. As seen in Fig. 2, the features exhibit a categorical distribution of values, which contributes to the high performance demonstrated by the CatBoost algorithm. Additionally, in the proposed approach, the CatBoost model is optimised with PSO to further enhance its performance.

5 Conclusion

This study proposed an effective machine learning model to estimate the number of k-barriers for robust intrusion detection and prevention for a rectangular area using features of the WSN. The features were obtained through the Monte-Carlo simulation process. The proposed CatBoost model, which is optimised with the PSO algorithm, estimated the number of barriers accurately. To ensure a fair assessment, the performance of the proposed model was compared with state-of-the-art methods such as LightGBM, XGBoost, RF, and DT using the MAE, MSE, RMSE, and R^2 metrics. The proposed model showed a clear superiority. Additionally, the performance of the proposed model was assessed against the other existing methods in the literature that use the same dataset, and it yielded the best results as well. Afterwards, the feature importance was analysed. The network area and transmission range appeared as the most relevant features for the proposed model.

The limitations of the proposed CatBoost-PSO approach can be grouped into theoretical and practical aspects. From a theoretical perspective, the CatBoost requires more computational resources during training compared to alternative models. Furthermore, the PSO is prone to getting stuck in local optimum points, especially in complex optimisation scenarios. Moreover, the effectiveness of PSO decreases in high-dimensional optimisation spaces due to the expansive search space, which limits the exploration capabilities of particles. From a practical perspective, the evaluation of the proposed model was limited to the rectangular network region of WSN, and it could be beneficial to explore its performance in the circular network region as well. Additionally, the proposed model was solely tested under the uniform distribution of sensors, whereas further research could include evaluating its effectiveness under the Gaussian distribution of sensors. These potential expansions present promising paths for future work and could enhance the versatility and robustness of the proposed model across various deployment scenarios.

In future work, the feature engineering techniques can also be used to enhance the predictive capability of the proposed model. These techniques include generating new features from existing features or transforming the existing features. Additionally, correlation analysis can be conducted to identify highly related features. Removing one of the identified highly correlated features can contribute to reducing the time complexity of the proposed model. Finally, surrogate-assisted optimization algorithms, such as Bayesian Optimization, could be implemented to reduce training time while maintaining reasonable detection accuracy.

Authors contributions Kadir Ileri wrote the whole manuscript, implemented machine learning algorithms, and finally reviewed the manuscript.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). Openaccess funding provided by the Scientific and Technological Research Council of Türkiye(TÜBİTAK). This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Ethical approval Ethics approval was not required for this study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Anders T (2020) Territorial control in civil wars: Theory and measurement using machine learning. *J Peace Res* 57(6):701–714. <https://doi.org/10.1177/0022343320959687>
- Singh A, Amutha J, Nagar J, Sharma S (2021) A deep learning approach to predict the number of k-barriers for intrusion detection over a circular region using wireless sensor networks. *Expert Syst. Appl.*, 211: 118588. <https://doi.org/10.1016/j.eswa.2022.118588>.
- Singh A, Amutha J, Nagar J, Sharma S, Lee CC (2022) AutoML-ID: automated machine learning model for intrusion detection using wireless sensor network. *Sci Rep* 12(1):1–14. <https://doi.org/10.1038/s41598-022-13061-z>
- de Campos Souza PV, Lughofer E, Rodrigues Batista H (2022) An explainable evolving fuzzy neural network to predict the k barriers for intrusion detection using a wireless sensor network. *Sensors* 22:14. <https://doi.org/10.3390/s22145446>.
- Nagar J, Chaturvedi SK, Soh S. (2022) Wireless multihop network coverage incorporating boundary and shadowing effects. *IETE Tech. Rev. (Institution Electron. Telecommun. Eng. India)* 39(5), 1124–1139. <https://doi.org/10.1080/02564602.2021.1968963>.
- Amutha J, Sharma S, Nagar J (2020) WSN strategies based on sensors, deployment, sensing models, coverage and energy efficiency: review, approaches and open issues. *Wirel Pers Commun* 111(2):1089–1115. <https://doi.org/10.1007/s11277-019-06903-z>
- Kandris D, Nakas C, Vomvas D, Koulouras G (2020) Applications of wireless sensor networks: An up-to-date survey. *Appl Syst Innov* 3(1):1–24. <https://doi.org/10.3390/asi3010014>

8. Si P, Ma J, Tao F, Fu Z, Shu L (2020) Energy-efficient barrier coverage with probabilistic sensors in wireless sensor networks. *IEEE Sens J* 20(10):5624–5633. <https://doi.org/10.1109/JSEN.2020.2970435>
9. Yang T, Mu D, Hu W, Zhang H (2014) Energy-efficient border intrusion detection using wireless sensors network. *Eurasip J Wirel Commun Netw* 2014:1–12. <https://doi.org/10.1186/1687-1499-2014-46>
10. Raza F, Bashir S, Tauseef K, Shah SI. Optimizing nodes proportion for intrusion detection in uniform and Gaussian distributed heterogeneous WSN. *Proc. 2015 12th Int. Bhurban Conf. Appl. Sci. Technol. IBCAST 2015*, pp. 623–628, 2015. <https://doi.org/10.1109/IBCAST.2015.7058571>.
11. Sharma S, Nagar J (2020) Intrusion detection in mobile sensor networks: a case study for different intrusion paths. *Wirel Pers Commun* 115(3):2569–2589. <https://doi.org/10.1007/s11277-020-07697-1>
12. Singh A, Nagar J, Sharma S, Kotiyal V. (2021) A Gaussian process regression approach to predict the k-barrier coverage probability for intrusion detection in wireless sensor networks. *Expert Syst. Appl.* 172, 114603. <https://doi.org/10.1016/j.eswa.2021.114603>.
13. Nurellari E, Licea DB, Ghogho M, Rivero-Angeles ME (2021) On trajectory design for intruder detection in wireless mobile sensor networks. *IEEE Trans Signal Inf Process over Networks* 7:236–248. <https://doi.org/10.1109/TSIPN.2021.3067305>
14. Saraereh OA, Ali A, Al-Tarawneh L, Khan I (2021) A robust approach for barrier-reinforcing in wireless sensor networks. *J Parallel Distrib Comput* 149:186–192. <https://doi.org/10.1016/j.jpdc.2020.12.007>
15. Amutha J, Nagar J, Sharma S (2021) A distributed border surveillance (DBS) system for rectangular and circular region of interest with wireless sensor networks in shadowed environments. *Wirel Pers Commun* 117(3):2135–2155. <https://doi.org/10.1007/s11277-020-07963-2>
16. M Sharma, CRS Kumar (2021) Machine learning-based smart surveillance and intrusion detection system for national geographic borders. In *Artificial Intelligence and Technologies: Select Proceedings of ICRTAC-AIT 2020*, pp. 165–176. Singapore: Sp, p. 2021.
17. Singh R, Singh S (2022) Smart border surveillance system using wireless sensor networks. *Int J Syst Assur Eng Manag* 13(s2):880–894. <https://doi.org/10.1007/s13198-021-01208-6>
18. Antony Joseph Rajan D, Gomathy CK. (2023) A robust intrusion detection mechanism in wireless sensor networks against well-armed attackers. *Int J Intell Syst Appl Eng* 11(2), 180–187.
19. Mahjoub W, Nakkach C, Ezzedine T (2023) Design of autonomous wireless sensor network using mobile robots for intrusion detection and border surveillance. *2023 Int Wirel Commun Mob Comput IWCMC 2023*:476–481. <https://doi.org/10.1109/IWCMC58020.2023.10183288>
20. Zhu Z, Pan Y, Zhou Q, Lu C (2021) Event-triggered adaptive fuzzy control for stochastic nonlinear systems with unmeasured states and unknown backlash-like hysteresis. *IEEE Trans Fuzzy Syst* 29(5):1273–1283. <https://doi.org/10.1109/TFUZZ.2020.2973950>
21. Roman RC, Precup RE, Petriu EM (2021) Hybrid data-driven fuzzy active disturbance rejection control for tower crane systems. *Eur J Control* 58:373–387. <https://doi.org/10.1016/j.ejcon.2020.08.001>
22. “Iot Communicaiton Protocols Accessed: 01.02.2024.” <https://research.aimultiple.com/iot-communication-protocol/>
23. Zungeru AM, Ang LM, Seng KP (2012) Classical and swarm intelligence based routing protocols for wireless sensor networks: A survey and comparison. *J Netw Comput Appl* 35(5):1508–1536. <https://doi.org/10.1016/j.jnca.2012.03.004>
24. “Dataset: Intrusion Detection in WSNs Accessed:20.07.2023.” <https://archive.ics.uci.edu/dataset/715/lt+fs+id+intrusion+detection+in+wsns>
25. Singh A, Amutha J, Nagar J, Sharma S, Lee CC (2022). LT-FS-ID: Log-transformed feature learning and feature-scaling-based machine learning algorithms to predict the k-barriers for intrusion detection using wireless sensor network. *Sensors*, 22(3). <https://doi.org/10.3390/s22031070>.
26. Aguinis H, Gottfredson RK, Joo H (2013) Best-practice recommendations for defining, identifying, and handling outliers. *Organ Res Methods* 16(2):270–301. <https://doi.org/10.1177/1094428112470848>
27. Hindrayani KM, Fahrudin TM, Prismahardi Aji R, Safitri EM (2020) Indonesian stock price prediction including Covid19 era using decision tree regression. *2020 3rd Int. Semin. Res. Inf. Technol. Intell. Syst. ISRITI 2020*, no. March 2020, pp. 344–347. <https://doi.org/10.1109/ISRITI51436.2020.9315484>.
28. Kale SJ, Khandekar MA, Agashe SD (2021) Prediction of water filter bed backwash time for water treatment plant using machine learning algorithm. *2021 IEEE 6th Int Conf Comput Commun Autom ICCCA 2021*:113–116. <https://doi.org/10.1109/ICCCA52192.2021.9666204>
29. Leo B (2001) Random Forests. *Mach Learn* 45:5–32
30. Maruf S, Javed K, Babri HA (2016) Improving text classification performance with random forests-based feature selection. *Arab J Sci Eng* 41(3):951–964. <https://doi.org/10.1007/s13369-015-1945-x>
31. Chen T, Guestrin C (2016) XGBoost: A scalable tree boosting system, *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, vol. 13–17-Aug, pp. 785–794. <https://doi.org/10.1145/2939672.2939785>.
32. Dong W, Huang Y, Lehane B, Ma G (2020) XGBoost algorithm-based prediction of concrete electrical resistivity for structural health monitoring. *Autom. Constr.* 114, 103155. <https://doi.org/10.1016/j.autcon.2020.103155>.
33. Gumus M, Kiran MS (2017) Crude oil price forecasting using XGBoost,. *2nd Int Conf Comput Sci Eng UBKM 2017*:1100–1103. <https://doi.org/10.1109/UBKM.2017.8093500>
34. Ke G *et al.* (2017) LightGBM: A highly efficient gradient boosting decision tree. *Adv. Neural Inf. Process. Syst.*, vol. 2017-Decem, no. Nips, pp. 3147–3155.
35. R. Jin, G. Agrawal (2003). Communication and Memory Efficient Parallel Decision Tree Construction, pp. 119–129. <https://doi.org/10.1137/1.9781611972733.11>.
36. Alsabti K, Ranka S. CLOUDS: A Decision Tree Classifier for Large Datasets. *AAAI Press*, vol. 2, pp. 2–8, 1998, [Online]. Available at <http://www.kdnuggets.com/>
37. Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A (2018) Catboost: Unbiased boosting with categorical features. *Adv. Neural Inf. Process. Syst.*, vol. 2018-Decem, no. Section 4, pp. 6638–6648.
38. Zhang K, Scholkopf B, Muandet K, Wang Z (2013), “Domain adaptation under target and conditional shift,” *30th Int. Conf. Mach. Learn. ICML 2013*, vol. 28, no. PART 3, pp. 1856–1864.
39. Kennedy J, Eberhart R (1995). Particle Swarm Optimization, in *In Proceedings of ICNN’95-international conference on neural networks*, pp. 1942–1948. https://doi.org/10.1007/978-3-319-46173-1_2.
40. Tang J, Liu G, Pan Q (2021) A review on representative swarm intelligence algorithms for solving optimization problems: applications and trends. *IEEE/CAA J Autom Sin* 8(10):1627–1643. <https://doi.org/10.1109/JAS.2021.1004129>
41. Habanik J, Grecikova A, Krajco K (2019) The impact of new technology on sustainable development. *Eng Econ* 30(1):41–49. <https://doi.org/10.5755/j01.ee.30.1.20776>

42. Xinchao Z (2010) A perturbed particle swarm algorithm for numerical optimization. *Appl Soft Comput J* 10(1):119–124. <https://doi.org/10.1016/j.asoc.2009.06.010>
43. Chen D, Wang J, Zou F, Zhang H, Hou W (2012) Linguistic fuzzy model identification based on PSO with different length of particles. *Appl Soft Comput J* 12(11):3390–3400. <https://doi.org/10.1016/j.asoc.2012.07.014>
44. Gad AG (2022) *Particle Swarm Optimization Algorithm and Its Applications: A Systematic Review*, vol. 29, no. 5. Springer Netherlands. <https://doi.org/10.1007/s11831-021-09694-4>
45. Shami TM, El-Saleh AA, Alswaitti M, Al-Tashi Q, Summakieh MA, Mirjalili S (2022) Particle swarm optimization: a comprehensive survey. *IEEE Access* 10:10031–10061. <https://doi.org/10.1109/ACCESS.2022.3142859>
46. Chicco D, Warrens MJ, Jurman G (2021) The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. *PeerJ Comput Sci* 7:1–24. <https://doi.org/10.7717/PEERJ-CS.623>
47. Haynatzki R, Windle T, Windle J (2023) Building machine learning models to improve hypertension diagnosis. *J Phys Conf Ser* 2675(1):012001. <https://doi.org/10.1088/1742-6596/2675/1/012001>
48. Singh J (2022) Computational complexity and analysis of supervised machine learning algorithms. *Next Generation of Internet of Things: Proceedings of ICNGIoT 2023*:195–206. https://doi.org/10.1007/978-981-19-1412-6_16
49. Arjun D, Indukala PK, Menon KAU (2017) Border surveillance and intruder detection using wireless sensor networks: A brief survey,” *Proc. 2017 IEEE Int. Conf. Commun. Signal Process. ICCSP 2017*, vol. 2018-Janua, pp. 1125–1130. <https://doi.org/10.1109/ICCSP.2017.8286552>
50. Ramesh MV (2009) Real-time wireless sensor network for landslide detection, *Proc. - 2009 3rd Int. Conf. Sens. Technol. Appl. SENSORCOMM 2009*, pp. 405–409. <https://doi.org/10.1109/SENSORCOMM.2009.67>
51. He Y-X, Lyu S-H, Jiang Y (2024), Interpreting deep forest through feature contribution and MDI feature importance,” *ACM Trans. Knowl. Discov. Data*, pp. 1–21. <https://doi.org/10.1145/3641108>
52. Tarafdar A et al (2024) Enhancing intrusion detection using wireless sensor networks: A novel ahp-madm aggregated multiple type 3 fuzzy logic-based k-barriers prediction system. *Peer-to-Peer Netw Appl* 17(3):1732–1749. <https://doi.org/10.1007/s12083-024-01688-w>
53. Singh A, Nagar J, Amutha J, Sharma S (2023) P2CA-GAM-ID: Coupling of probabilistic principal components analysis with generalised additive model to predict the k-barriers for intrusion detection, *Eng. Appl. Artif. Intell.*, vol. 126, no. September. <https://doi.org/10.1016/j.engappai.2023.107137>
54. Roy K, Das RN, Ambure P, Aher RB (2016) Be aware of error measures. Further studies on validation of predictive QSAR models. *Chemom Intell Lab Syst* 152:18–33. <https://doi.org/10.1016/j.chemolab.2016.01.008>
55. Rondinella F, Daneluz F, Hofko B, Baldo N (2023) Improved predictions of asphalt concretes’ dynamic modulus and phase angle using decision-tree based categorical boosting model, *Constr. Build. Mater.*, vol. 400, no. August, p. 132709. <https://doi.org/10.1016/j.conbuildmat.2023.132709>
56. Gabriel JJ, Anbarasi LJ (2023) Optimizing coronary artery disease diagnosis: a heuristic approach using robust data preprocessing and automated hyperparameter tuning of eXtreme gradient boosting. *IEEE Access* 11(August):112988–113007. <https://doi.org/10.1109/ACCESS.2023.3324037>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.